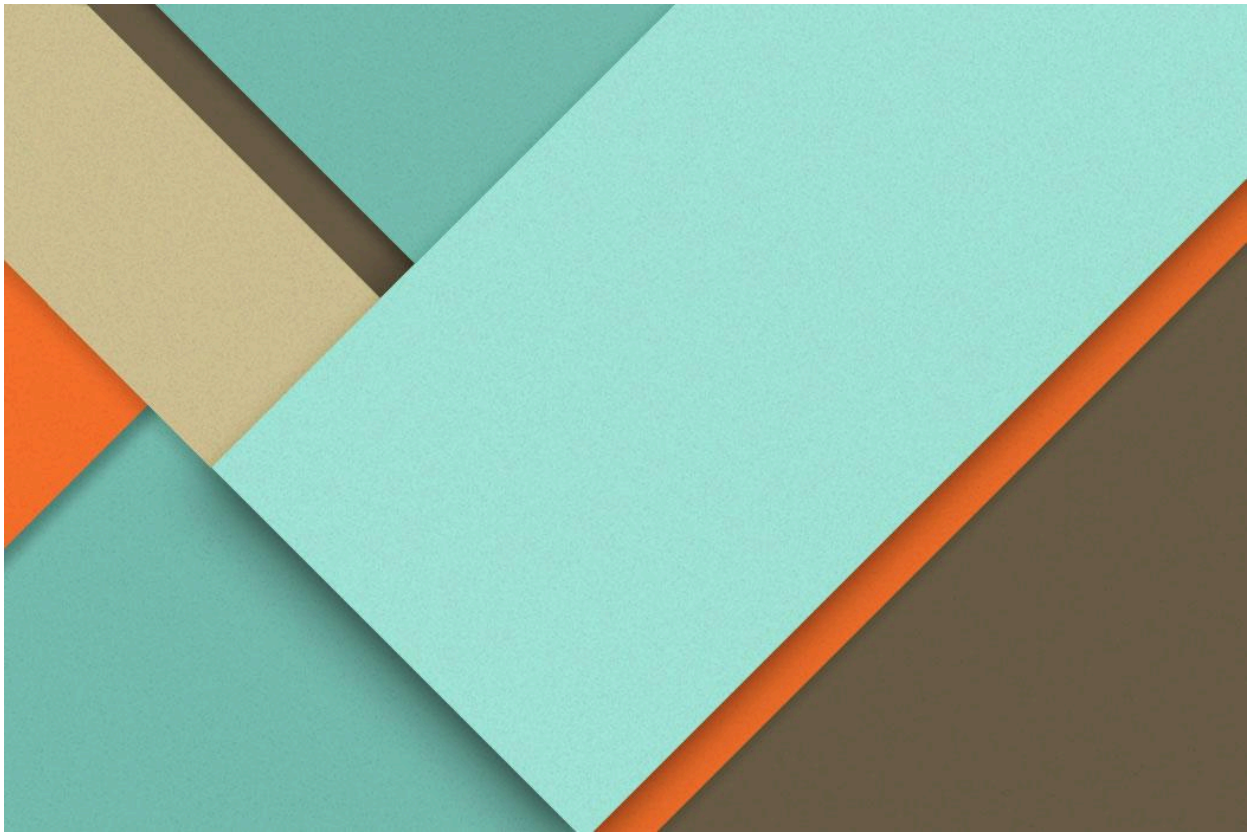
1.



Implementación distribuida del algoritmo de Mandelbrot usando Kubernetes

18/02/2026

Diego Josue Avila Ochoa
Angel Yael Gonzales Lorenzana
Jorge Mario Martínez Pérez
Peña Lara Ian Paul Alejandro
Peralta Fonseca Juan José

Introducción

Se desarrollara un cluster con el objetivo de correr un algoritmo, todo esto para poder demostrar el uso y potencial de las tecnologías paralelas, lográndolo por medio de tecnologías como lo son las máquinas virtuales, VPNs y contenedores, corriendo así el algoritmo de Mandelbrot, para demostrar cómo el tener varias computadoras trabajando a la vez puede mejorar el rendimiento en trabajos donde se permite demostrar esto.

Marco teórico

Máquina virtual

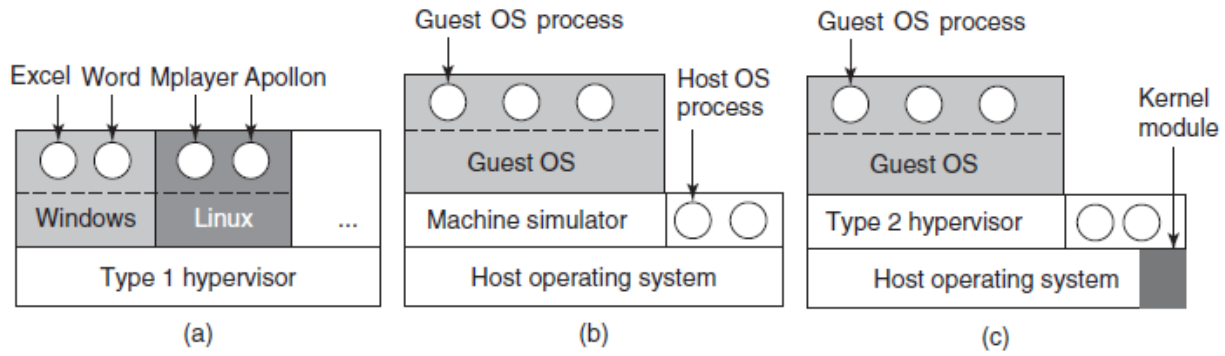
Una máquina virtual es una réplica del hardware físico creada por un software que se ejecuta directamente sobre la máquina real. Este software es conocido como monitor de máquina virtual o **hypervisor**, administra los recursos del sistema y permite que múltiples sistemas operativos funcionen de manera simultánea en una máquina.

Una máquina virtual reproduce las características del equipo subyacente: modos de ejecución (usuario y kernel), manejo de interrupciones, dispositivos de entrada y salida, y demás mecanismos propios del hardware real. Cada máquina virtual puede ejecutar cualquier sistema operativo diseñado para la arquitectura física original, como si estuviera instalada en un equipo independiente. Por ejemplo, se puede virtualizar Debian x86 en un ordenador con un procesador x86.

Cada entorno virtual opera de forma aislada. Cuando un programa dentro de una máquina virtual realiza una llamada al sistema, esta es atendida por el sistema operativo que se ejecuta dentro de esa misma máquina virtual. Si se requiere una operación privilegiada, como acceso a dispositivos de entrada/salida, el hipervisor intercepta la instrucción y la gestiona, simulando el comportamiento del hardware real.

Este mecanismo permite mantener la separación entre entornos y garantiza que el funcionamiento de uno no interfiera con los demás.

Tipos de hypervisor



(a) Hipervisor Tipo 1 (bare-metal)

El hipervisor se ejecuta directamente sobre el hardware físico y controla el CPU, memoria y dispositivos. Encima del hipervisor se ejecutan múltiples sistemas operativos invitados. Por ejemplo, Hyper-V ejecuta windows como una máquina virtual privilegiada llamada root partition y las demás máquinas se ejecutan como *child partitions*. Lo atractivo de este tipo de hipervisor es el mayor rendimiento y menor sobrecarga, pues muchas instrucciones pueden ejecutarse directamente en el procesador, aunque las instrucciones privilegiadas siguen siendo gestionadas por el hipervisor por razones de seguridad y aislamiento.

(b) Hipervisor Tipo 2 "puro"

El hipervisor se ejecuta totalmente como una aplicación sobre un sistema operativo anfitrión. El hipervisor corre como un proceso más del host y dentro del hipervisor se ejecuta el sistema operativo invitado. La ventaja que dan es que son más fáciles de instalar, sin embargo, en cuestión de rendimiento están por detrás de los tipo 1. Este tipo es más teórico que práctico y por ende no tiene aplicaciones reales serias.

(c) Hipervisor Tipo 2 “práctico”

Aunque **el hipervisor corre sobre el sistema operativo anfitrión**, parte de **su funcionalidad crítica se implementa como módulos del kernel del host.** En el mundo comercial esta implementación es estándar, ejemplos de este tipo son Oracle VM VirtualBox y VMware workstation. Este tipo es más deseable que el tipo 2 puro pues reduce la sobrecarga y mejora el rendimiento al ejecutar partes en modo kernel.

fuente: Tanenbaum (2001)

Red Privada Virtual (VPN)

Una Red Privada Virtual, o VPN por sus siglas en inglés, es un mecanismo de red el cual tiene como objetivo conectar de manera segura dos o más puntos a través de una red pública (como el internet), por medio de encapsulación de paquetes, encriptación y autenticación criptográfica. Trabaja en las capas 2 y 3 del Modelo OSI.

En el contexto de nuestro proyecto la VPN se usa para conectar nuestros equipos a través del internet para poder realizar la configuración de los nodos de kubernetes que se usarán para ejecutar el algoritmo seleccionado. La VPN que se usó es WireGuard, por lo cual fue necesario configurar cada uno de los nodos de manera manual, donde se declara quién es el servidor y quien es el cliente, esto es importante para nuestra topología, hub and spoke, la cual se describe más adelante.

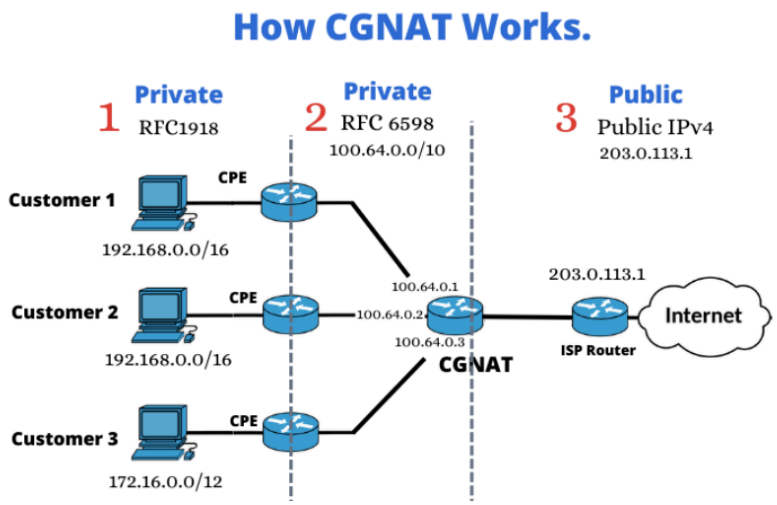
Para asegurar que se realizará una conexión exitosa entre la red, tenemos que asegurar que fuéramos accesible de manera pública y que estuviéramos escuchando los mensajes que enviamos, por eso es que se hizo un port forward del puerto 51820 y asegurarnos que tuviéramos una IP pública, para evitar problemas de CGNAT.

CGNAT

Según la Internet Engineering Task Force (2011), el CGNAT (Carrier-Grade NAT) es un mecanismo implementado por los proveedores de servicios de Internet (ISP) para mitigar la escasez de direcciones IPv4, mediante el cual u**na dirección IPv4 pública**

es compartida por múltiples suscriptores. En este esquema, a cada usuario se le asigna una dirección IP privada y un traductor de direcciones de red (NAT) ubicado dentro de la infraestructura del ISP realiza la traducción entre direcciones privadas y públicas. Cuando además existe un segundo NAT en el borde del cliente, se produce un escenario de doble traducción (doble NAT).

Funcionamiento de CGNAT



La imagen representa la estructura de CGNAT, entre el internet y los clientes se coloca una red privada que cumple la función de NAT, es decir traducir la dirección del cliente a otra dentro de la red privada. CGNAT se almacena una tabla que relaciona la ip privada del cliente y su puerto con una ip pública compartida y un puerto aleatorio. Cuando el destino responda a través del mismo puerto, CGNAT sabrá a qué cliente enrutar el tráfico.

Ventajas y desventajas de CGNAT

La ventaja de CGNAT es que combate la escasez de ipv4. Sin embargo, esto viene con el costo de que hace imposible el uso de aplicaciones que requieren puertos específicos (instalar un servidor web, por ejemplo), ya que al intentar conectarse

utilizando la supuesta dirección pública del servidor, la CGNAT no sabrá a qué cliente enrutar el tráfico y este será descartado .

Conjunto de mandelbrot

El conjunto de mandelbrot es fractal que está definido en el plano complejo como el conjunto de los puntos c, este algoritmo es perfecto para la propuesta del proyecto, ya que es un algoritmo recursivo que necesita mucha potencia computacional distribuida para llevarlo a gran escala.

El conjunto de Mandelbrot se define por la iteración de la función:

$$z_{n+1} = z_n^2 + c, \text{ con } z_0 = 0$$

Las ecuaciones que son necesarias para ser computadas, las cuales son las siguientes:

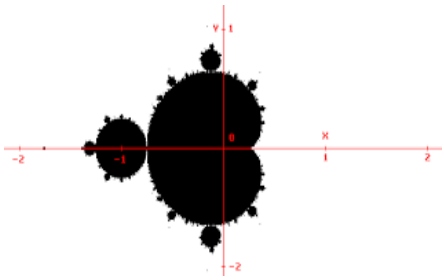
$$z_{n+1} = z_n^2 + c \qquad x_{n+1} = x_n^2 - y_n^2 + c_r \qquad y_{n+1} = 2x_n y_n + c_i$$

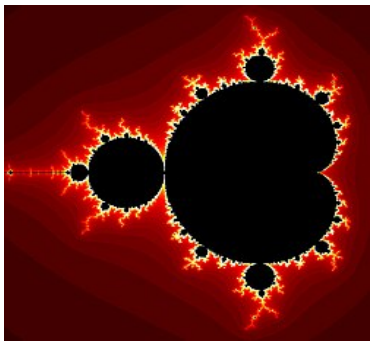
z: Donde si la solución |z| no escapa al conjunto,”c” pertenece al conjunto y se coloca su valor como pixel.

n: Número de iteraciones

c: Es la variable de posición y es un número complejo (c=a+bi) que representa cada píxel en el plano complejo. Es la variable fundamental que determina si un punto pertenece al conjunto o no.

“x” y “y”: Son los ejes del plano





El cálculo del conjunto de Mandelbrot tiene una complejidad aproximada de:

$O(W \times H \times N)$

Donde:

- W, H: resolución de la imagen
- N: número máximo de iteraciones

Al aumentar la resolución o el nivel de zoom, el tiempo de cómputo crece de manera exponencial, lo que hace inviable su cálculo en un solo nodo para imágenes de alta precisión.

El plano complejo puede dividirse en regiones (tiles o bloques), donde cada nodo trabajador procesa un subconjunto de píxeles.
Cada worker recibe:

- Coordenadas del plano $(x_{min}, x_{max}, y_{min}, y_{max})$
- Resolución
- Número máximo de iteraciones

El nodo devuelve un buffer de valores de iteración que posteriormente es ensamblado por el nodo central.

El **Hub (Master)**:

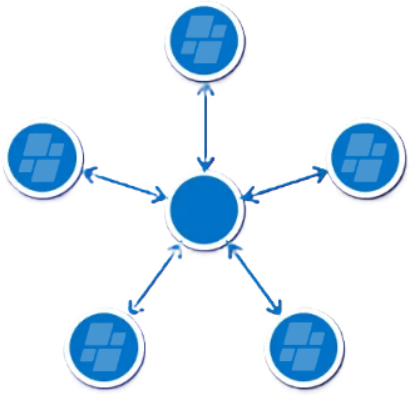
  - Divide el plano complejo en regiones.
  - Asigna tareas a los workers.
  - Recopila los resultados.
  - Genera la imagen final.

Los **Spokes (Workers)**:

  - Ejecutan el cálculo del Mandelbrot sobre su región asignada.
  - Devuelven únicamente los resultados al Hub.

Al mapear el plano complejo en una imagen, cada píxel es independiente, esto permite que distribuir las regiones entre varias computadoras sea considerado como un buen escenario para una práctica de programación paralela y distribuida.

Topología *hub and spoke*



La topología Hub-and-Spoke es un modelo de red donde todos los nodos periféricos (Spokes) se conectan exclusivamente a un nodo central (Hub). En nuestro proyecto, esta arquitectura es fundamental por las siguientes razones:

- **Centralización de la VPN:** El nodo Hub (ubicado en la VM del integrante Yael Lorenzana) actúa como el servidor central de WireGuard, gestionando las llaves públicas de todos los integrantes y permitiendo que los nodos se descubran entre sí.
- **Superación de CGNAT y Doble NAT:** Dado que la mayoría de los integrantes se encuentran detrás de CGNAT, lo cual impide recibir conexiones entrantes, la topología permite que solo un nodo (el Hub) necesite realizar Port Forwarding para recibir las conexiones de los demás.
- **Gestión del Clúster de Kubernetes:** Esta topología se alinea perfectamente con la arquitectura de k3s, donde el nodo central asume el rol de Control-Plane y los nodos periféricos actúan como Agentes (Workers).
- **Flujo de Datos del Algoritmo:** Para el cálculo del conjunto de Mandelbrot, el Hub funciona como el Coordinador que segmenta el plano complejo y distribuye las tareas a los Spokes, quienes procesan los datos y devuelven los resultados al centro para la reconstrucción de la imagen final.

Rol	Integrante / Nodo	Dirección IP VPN
Hub / Control-Plane	Yael Lorenzana	10.10.10.1
Spoke / Worker 1	Ian Paul Peña	10.10.10.2
Spoke / Worker 2	Juan José Peralta	10.10.10.3
Spoke / Worker 3	Jorge Martínez	10.10.10.4
Spoke / Worker 4	Josue Avila	10.10.10.5

Arquitectura General del Sistema

El proyecto debido al contexto de su desarrollo fue necesario implementar un entorno basado en la topología hub-and-spoke, constando de un nodo central y 4 nodos conectados al mismo, administrados mediante kubernetes (k3s). A cada nodo se le deben asignar 4 pods para la distribución de carga del proceso del algoritmo implementado, teniendo un total de 20 pods (o réplicas) tomando en cuenta al dispositivo que funciona como control-plane con un rol híbrido, donde no solo le indica le asigna el rol de worker a los nodos que administra, sino que también forma parte del proceso de trabajo , además de la integración final, recordemos el algoritmo planteado de momento es el Mandelbrot, por lo que se necesita integral los resultados en una imagen final.

Objetivo

Objetivo Principal:

Crear un cluster diseñado para correr programas altamente escalables en un entorno de contenedores dentro de máquinas virtuales de Linux, por medio de una VPN.

Objetivos Particulares:

- Diseñar, configurar y validar una red VPN privada entre los integrantes del equipo
- Preparar la infraestructura básica de contenedores Docker/Kubernetes que se utilizará posteriormente para…
- Ejecutar un problema computacional basado en el conjunto de mandelbrot implementado en Rust de forma distribuida.

Desarrollo

El desarrollo del proyecto hasta el momento consistió en 3 etapas iniciales:

Acondicionamiento de dispositivos: En esta etapa se instalaron las herramientas iniciales para empezar con el desarrollo del proyecto, se instaló Hyper-V para poder realizar la virtualización, la cual se cargó con una imagen con el SO Ubuntu Server, y se le realizó la asignación de recursos y modificaciones de periféricos (interfaz de red, preferencias, etc.) pertinentes.

Creación de Red Virtual Privada (VPN): En esta fase se creó una vpn con la intención de poder interconectar los dispositivos de los integrantes del equipo en un entorno seguro, para esto fue necesario hacer uso de Port Forwarding (PF), en la cual requirió de la configuración de un puerto en el router de uno de los integrantes del equipo, para esto fue necesario elegir realizarlo en el de un integrante en específico debido a las limitaciones que conlleva hacer uso del PF, la cual en este caso fue que no todas las compañía de red dan las facilidades para acceder a la configuración del router, y

debido a contar con un servicio de Internet que permitió realizar dichos ajustes, fue que se seleccionó la máquina del Integrante Yael Lorenzana, más allá de las especificaciones (Potencia) de su dispositivo.

Levantar Kubernetes: En la última de estas primeras etapas, para el uso de kubernetes se usó k3s, con ello se ejecutó un archivos docker-deployment.yml previamente desarrollado, con las indicaciones para la creación de 20 réplicas, que realizarán la misma tarea, realizar un seguimiento en los logs de un documento NULL, con la intención de que se no se fueran a detener al ejecutar una tarea “finita”, con lo que se comprobo la correcta instalacion y distribucion de procesos entre los pods.

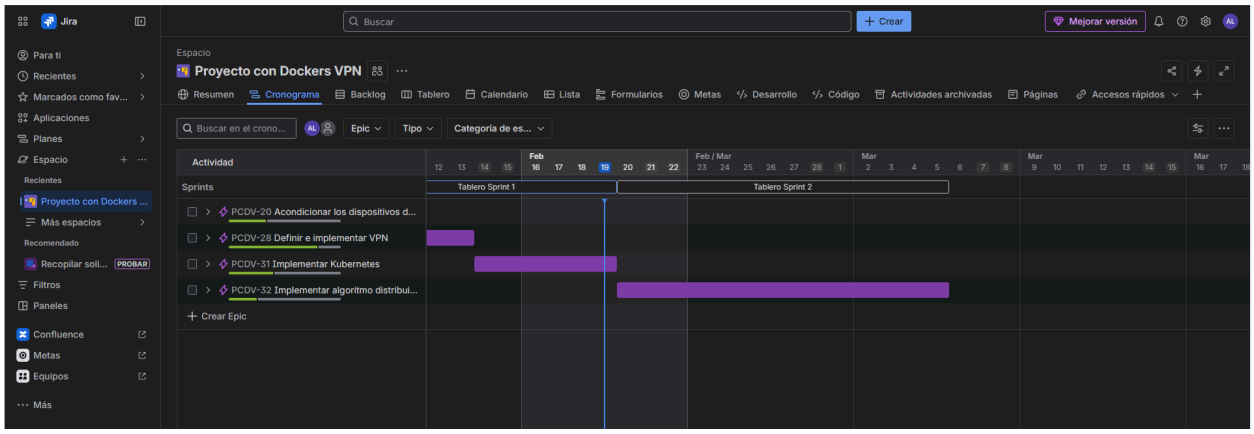
Code Blame 26 lines (23 loc) · 505 Bytes

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sistema-distribuido
spec:
  replicas: 20
  selector:
    matchLabels:
      app: worker
  template:
    metadata:
      labels:
        app: worker
    spec:
      containers:
      - name: worker-node
        image: yaellorenzana/worker-node:v1
        imagePullPolicy: Always


        command: ["/bin/sh", "-c", "tail -f /dev/null"]

        resources:
          limits:
            memory: "128Mi"
            cpu: "200m"
```

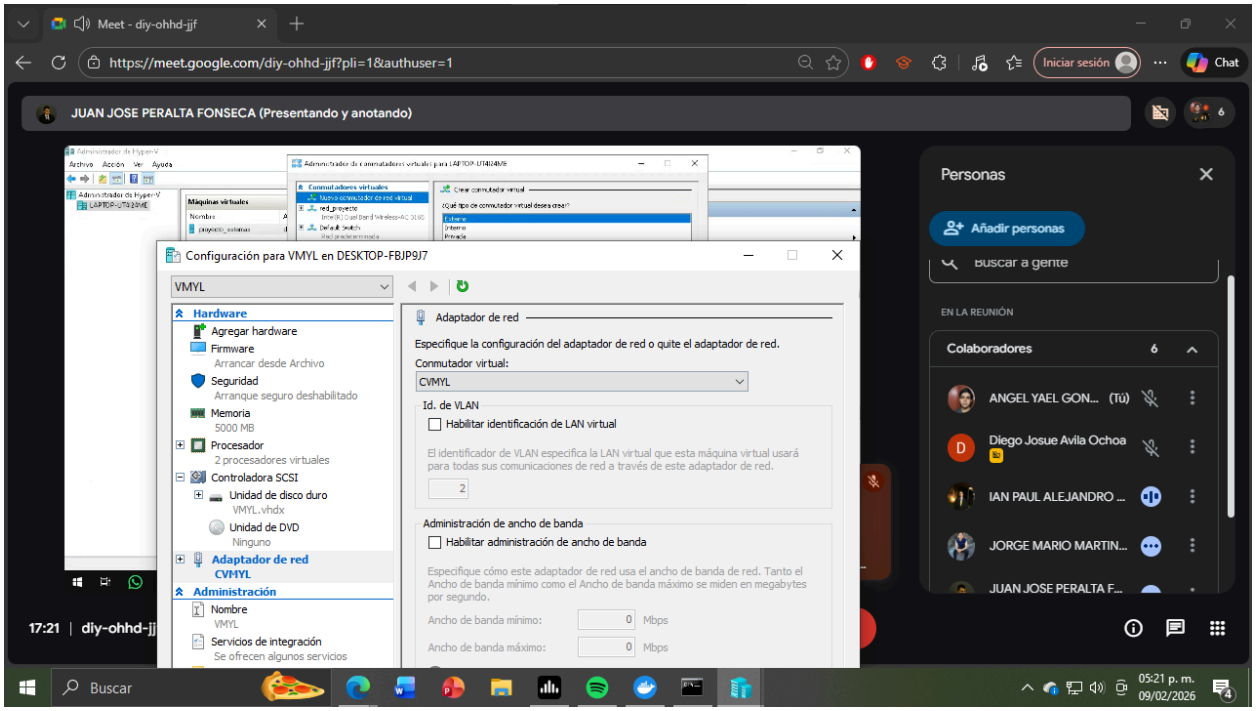
```
NAME                                    READY   STATUS    RESTARTS   AGE   IP          NODE       NOMINATED NODE   READINESS G
ATES
sistema-distribuido-5dbb8f77f-2pt8q     1/1     Running   0          72s   10.42.2.24  ianserver  <none>           <none>
sistema-distribuido-5dbb8f77f-2q4nh     1/1     Running   0          72s   10.42.3.14  wgserver   <none>           <none>
sistema-distribuido-5dbb8f77f-4n7hx     1/1     Running   0          72s   10.42.4.12  vmja       <none>           <none>
sistema-distribuido-5dbb8f77f-54z8d     1/1     Running   0          72s   10.42.0.60  vmyl       <none>           <none>
sistema-distribuido-5dbb8f77f-58v7j     1/1     Running   0          72s   10.42.4.13  vmja       <none>           <none>
sistema-distribuido-5dbb8f77f-6z48q     1/1     Running   0          72s   10.42.3.12  wgserver   <none>           <none>
sistema-distribuido-5dbb8f77f-cqjkh     1/1     Running   0          72s   10.42.4.11  vmja       <none>           <none>
sistema-distribuido-5dbb8f77f-cr569     1/1     Running   0          72s   10.42.1.17  jorge      <none>           <none>
sistema-distribuido-5dbb8f77f-d4bgc     1/1     Running   0          72s   10.42.2.21  ianserver  <none>           <none>
sistema-distribuido-5dbb8f77f-dhcw2     1/1     Running   0          72s   10.42.1.16  jorge      <none>           <none>
sistema-distribuido-5dbb8f77f-hhq6z     1/1     Running   0          72s   10.42.0.61  vmyl       <none>           <none>
sistema-distribuido-5dbb8f77f-n7sxl     1/1     Running   0          72s   10.42.2.22  ianserver  <none>           <none>
sistema-distribuido-5dbb8f77f-s67xm     1/1     Running   0          72s   10.42.4.14  vmja       <none>           <none>
sistema-distribuido-5dbb8f77f-sbn5l     1/1     Running   0          72s   10.42.3.11  wgserver   <none>           <none>
sistema-distribuido-5dbb8f77f-sg4qd     1/1     Running   0          72s   10.42.0.58  vmyl       <none>           <none>
sistema-distribuido-5dbb8f77f-sjwqk     1/1     Running   0          72s   10.42.2.23  ianserver  <none>           <none>
sistema-distribuido-5dbb8f77f-spgdh     1/1     Running   0          72s   10.42.3.13  wgserver   <none>           <none>
sistema-distribuido-5dbb8f77f-sppgv     1/1     Running   0          72s   10.42.4.10  vmja       <none>           <none>
sistema-distribuido-5dbb8f77f-w9jrx     1/1     Running   0          72s   10.42.1.18  jorge      <none>           <none>
sistema-distribuido-5dbb8f77f-wnpw6     1/1     Running   0          72s   10.42.0.59  vmyl       <none>           <none>
```

Planeación

El proyecto se llevó utilizando la metodología ágil SCRUM, para la generación del Backlog, Sprint Backlog, se utilizó la herramienta de Jira la cual facilita la construcción del cronograma en base al backlog, siempre y cuando se le de la información correspondiente.



Los sprints que se realizaron al momento fueron tres, cada uno correspondiente a cada una de las etapas mencionadas anteriormente en. Los meetings se llevaron a cabo en línea mediante Google Meet, siendo igualmente 3 de estos, de los cuales las minutas fueron recabadas y recopiladas en un documento de excel para llevar mejor control de estos.



Entorno de desarrollo

Como primera opción se consideró utilizar WSL2, se evaluaron los siguientes puntos:

Ventajas

1. Fácil de instalar
2. Ligero
3. Reproducible

Desventajas

1. El tráfico de red de WSL2 es manejado internamente por Windows. Significa que no tiene una ip visible en la LAN del equipo físico y a su vez implica que el OS host e invitado comparten la misma ip pública.

   Lo anterior representa un problema por dos razones, la comunicación con el nodo central no es directa; pues el tráfico primero debe pasar por el OS anfitrión, Windows y este debe delegarlo a su instancia de WSL2. Para esto, se requiere configurar el *firewall* de Windows para realizar *port forwarding.* La capa extra que

representa Windows puede generar problemas, inconvenientes e inconsistencias.

La segunda razón, estrechamente relacionada a la primera, es que los requerimientos del proyecto solicitan el uso de herramientas como *ip tables.* Como ya se dijo, el tráfico de red es manejado por windows, por lo que el firewall "verdadero" es el firewall de windows.

Por esta razón fue que se eligió **Hyper-V** como hypervisor y ubuntu server como instancia. El cual otorga las siguientes ventajas:

1. No existe un sistema operativo intermedio : hyper-V es un hypervisor nivel 1 que permite ejecutar máquinas virtuales en el hardware. Lo cual habilita aspectos como que las reglas del firewall aplicadas a través de *ip tables* sean efectivas
2. Networking real: Es posible configurar un switch virtual externo en hyper-V para que las máquinas virtuales tengan ips visibles en la LAN del equipo. No obstante, esto aún no soluciona la accesibilidad al servidor, pues puede que el nodo se encuentre detrás de CGNAT.
3. Aislamiento:a diferencia de WSL2, el sistema de archivos de las máquinas virtuales en hyper-v no son directamente accesibles, cada vm tiene su disco virtual separado y también cuentan con adaptadores de red dedicados.

Wireguard

WireGuard se define como un túnel de red seguro que opera en la capa 3 del modelo OSI, implementado originalmente como una interfaz de red virtual para el kernel de Linux. A diferencia de protocolos tradicionales como IPsec o OpenVPN, WireGuard se diseñó con el objetivo de ser más sencillo de configurar, más rápido en términos de rendimiento y más fácil de auditar debido a que su código es relativamente pequeño.

Cryptokey Routing

El concepto central de WireGuard es el Cryptokey Routing. Este mecanismo asocia llaves públicas criptográficas con una lista de direcciones IP permitidas dentro del túnel.

- **Identificación:** Cada par (*peer*) se identifica únicamente por su llave pública, similar al funcionamiento de SSH.
- **Seguridad de Red:** Al recibir un paquete, la interfaz verifica que la llave pública del emisor coincida con su dirección IP de origen; si no hay una correspondencia exacta en la tabla de enrutamiento criptográfico, el paquete se descarta inmediatamente. Esto elimina la necesidad de reglas de firewall complejas para la autenticación básica de nodos.

Configuración Técnica y Herramientas

Para que la interfaz de WireGuard funcionara correctamente, seguimos la arquitectura hub-and-spoke (Explicada es su correspondiente apartado del marco teórico), además se tuvieron que seguir los siguientes requisitos técnicos:

1. **Generación de llaves:** Cada nodo del equipo generó sus propios pares de llaves criptográficas (pública y privada).
2. **Archivo de Configuración:**

```
[Interface]
PrivateKey = <LLAVE_PRIVADA_DEL_PEER>
Address = 10.0.0.2/24
DNS = 1.1.1.1

[Peer]
PublicKey = <LLAVE_PUBLICA_DEL_SERVIDOR>
Endpoint = IP_PUBLICA_DEL_SERVIDOR:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Plantilla peer

```
[Interface]
PrivateKey = <LLAVE_PRIVADA_DEL_SERVIDOR>
Address = 10.0.0.1/24
ListenPort = 51820

[Peer]
PublicKey = <LLAVE_PUBLICA_DEL_PEER>
AllowedIPs = 10.0.0.2/32
```

Plantilla servidor

3. **Direccionamiento IP:** Red 10.10.10.0/24, asignando la IP 10.10.10.1 al Hub y las IPs subsecuentes (.2, .3, .4, y .5) a los Peers.
4. **Herramientas Obligatorias:** Uso de comandos como wg, wg-quick, ip, ping, e iperf3 fueron necesarios para la gestión y validación de la conectividad.

```
root@vmyl:/etc/wireguard# sudo wg show
interface: wg0
  public key: nuEA6qk+1MFQpYHYXxtOr5mKpr11ep2ez83apY7EzTg=
  private key: (hidden)
  listening port: 51820

peer: QYjgeR6exfbadfzlFaFGfHWcHRfcpDofYDRqhOHJolY=
  endpoint: 177.245.203.54:42648
  allowed ips: 10.100.0.4/32
  latest handshake: 19 seconds ago
  transfer: 3.48 KiB received, 2.16 KiB sent

peer: P3izI8I5Q8SVhcxh9Fjq6ci9WQ5MWIzjW5CkMeu4v0M=
  endpoint: 187.188.64.83:51820
  allowed ips: 10.100.0.3/32
  latest handshake: 21 seconds ago
  transfer: 912 B received, 1.30 KiB sent

peer: K3NTEVUZMf5lvbVzJqFrFQlOmGRbba+qNBr6G5Ietyc=
  endpoint: 189.203.207.120:51820
  allowed ips: 10.100.0.2/32
  latest handshake: 49 seconds ago
  transfer: 534.62 KiB received, 522.30 KiB sent
```

Conexión exitosa entre servidor y peer's

Conclusión

Al terminar el primer avance, el equipo fue capaz de crear toda la infraestructura necesaria para ejecutar el algoritmo distribuido. Los problemas encontrados estuvieron relacionados principalmente con la conectividad cliente-servidor, la elección del entorno de desarrollo y la falta de experiencia con ciertas herramientas por parte del equipo.

Referencias

Internet Engineering Task Force. (2011, octubre). NAT444: Issues with IP address sharing (LSN/CGN requirements) (Internet-Draft draft-ietf-behave-lsn-requirements). https://datatracker.ietf.org/doc/html/draft-ietf-behave-lsn-requirements.

Andrew S. Tanenbaum. (2001). Modern operating systems (2nd ed., pp. 69–71). Prentice Hall.

Welcome! (s. f.). Minikube. https://minikube.sigs.k8s.io/docs/

Docker Hub Container Image Library | App Containerization. (s. f.). https://hub.docker.com/

Donenfeld, J. A. (2017). WireGuard: Next Generation Kernel Network Tunnel. En Proceedings of the Network and Distributed System Security Symposium (NDSS). San Diego, CA. https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/wireguard-next-generation-kernel-network-tunnel/